

ОТЧЁТ (30 вариант)

Назначение программы:

Программа реализует обработку одномерного массива согласно варианту 30, т.е. формирует массив В из элементов массива А, которые больше всех предыдущих элементов.

Модульная структура (5 файлов):

- `main.asm` - точка входа, организует выполнение тестов и интерактивный режим работы с файлами
- `macrolib.asm` - библиотека макросов-оберток для упрощения вызовов подпрограмм и системных функций
- `array_processing.asm` - реализация алгоритма формирования массива В согласно варианту 30
- `io.asm` - модуль ввода-вывода: работа с файлами, парсинг данных, форматированный вывод массивов
- `test.asm` - автоматизированное тестирование с набором тест-кейсов для проверки корректности работы программы

Режим работы:

1. Автоматическое тестирование
2. Интерактивная обработка файлов

Описание каждого файла:

main.asm

Главный управляющий модуль, который запускает автоматические тесты и переходит в интерактивный режим работы с файлами.

main - точка входа программы:

Запускает автоматические тесты через макрос `RUN_TESTS`

Выводит сообщение о переходе в пользовательский режим

Передаёт управление в `interactive_mode`

interactive_mode - основной интерактивный цикл:

Выводит приглашение "Enter file path: "

Полностью очищает буфер ввода (записывает нули во все 256 байт)

Читает путь к файлу от пользователя

Удаляет символ новой строки из ввода

Вызывает `read_file` для чтения файла

Вызывает `parse_file_data` для разбора данных

Проверяет корректность размера массива (1-10 элементов)

file_error - обработчик ошибок файлового ввода:

Выводит сообщение "ERROR: The file cannot be opened!"

Полностью очищает `file_buffer` (256 байт)

Возвращает управление в `interactive_mode` для повторной попытки

size_error - обработчик ошибок размера массива:

Выводит сообщение "ERROR: Invalid array size (allowed size 1-10)!"

Полностью очищает `file_buffer`

Возвращает управление в `interactive_mode`

process_arrays - основной процессор данных:

Выводит "N = " и размер массива

Выводит "A: " и содержимое массива A через PRINT_ARRAY

Преобразует массив A в B через FORM_ARRAY_B

Сохраняет размер массива B в память

Выводит "B: " и содержимое массива B

exit - завершение программы:

Вызывает макрос EXIT для корректного завершения работы

Дополнительные процедуры очистки:

- clear_buffer - очистка основного буфера ввода
- clear_file_buffer - очистка файлового буфера после ошибки файла
- clear_after_size_error - очистка файлового буфера после ошибки размера

macrolib.asm

Библиотека макросов, предоставляющая упрощенные интерфейсы для вызова подпрограмм и системных вызовов.

Макросы-обертки для пользовательских подпрограмм:

REMOVE_NEWLINE(%str) - удаляет символ новой строки из строки:

Загружает адрес строки в a0

Вызывает remove_newline

PRINT_ARRAY(%arr, %size) - форматированный вывод массива:

Загружает адрес массива в a0

Загружает размер массива в a1

Вызывает print_array

FORM_ARRAY_B(%arrA, %arrB, %sizeA) - формирует массив B из A по варианту 30:

Загружает адреса массивов A и B в a0 и a1

Загружает размер массива A в a2

Вызывает build_array_B

Возвращает размер массива B в a0

RUN_TESTS() - запуск полного набора автоматических тестов:

Вызывает run_tests без параметров

RUN_TEST(%arrA, %sizeA, %arrB, %expected_size) - выполнение одиночного теста:

Загружает тестовые данные: массив A, его размер, ожидаемый массив B и ожидаемый размер

Вызывает run_test

Макросы системных вызовов:

PRINT_STR(%str) - вывод строки в консоль:

Загружает адрес строки в a0

Устанавливает код системного вызова 4 (print string)

Выполняет ecall

PRINT_INT(%reg) - вывод целого числа:

Копирует значение из регистра в a0

Устанавливает код системного вызова 1 (print integer)

Выполняет ecall

INPUT_STR(%buf, %size) - ввод строки с клавиатуры:

Загружает адрес буфера в a0

Загружает размер буфера в a1

Устанавливает код системного вызова 8 (read string)

Выполняет `escall`

`EXIT()` - завершение программы:

Устанавливает код системного вызова 10 (`exit`)

Выполняет `escall`

array_processing.asm

Модуль реализации алгоритма по варианту 30, формирует массив `B` из элементов `A`, которые больше всех предыдущих элементов.

Основной метод:

build_array_B - формирует массив `B` из элементов `A[i]`, которые больше всех предыдущих элементов `A[j]` (где $j < i$):

Параметры:

- `a0` - указатель на исходный массив `A`
- `a1` - указатель на результирующий массив `B`
- `a2` - размер массива `A`

Возвращает:

- `a0` - размер созданного массива `B`

Внутренняя структура метода:

Пролог - инициализация стека:

Выделяет 28 байт (7 слов) на стеке

Сохраняет регистры `ra`, `s0-s5`

Инициализация - настройка рабочих переменных:

- `s0` - базовый адрес массива `A`
- `s1` - базовый адрес массива `B`
- `s2` - размер массива `A`
- `s3` - размер массива `B` (инициализируется 0)
- `s4` - счетчик внешнего цикла (`i`)
- `s5` - счетчик внутреннего цикла (`j`)

element_loop - внешний цикл по всем элементам `A`:

Проверяет границы ($i < \text{size_A}$)

Инициализирует флаг `t0 = 1` (предполагает выполнение условия)

compare_previous_loop - внутренний цикл сравнения:

Для каждого `j` от 0 до `i-1`

Загружает `A[i]` в `t2` и `A[j]` в `t4`

Сравнивает `A[i] > A[j]`

Если условие нарушено, устанавливает флаг `t0 = 0` и выходит из цикла

check_flag - проверка результата сравнения:

Если флаг `t0 = 1`, элемент удовлетворяет условию

Вычисляет адрес `A[i]` и `B[size_B]`

Копирует `A[i]` в `B[size_B]`

Увеличивает `size_B` на 1

next_element - переход к следующему элементу:

Увеличивает счетчик `i` на 1

Возвращается к началу внешнего цикла

return_result - завершение работы:

Копирует размер массива `B` в `a0` для возврата

Восстанавливает сохраненные регистры

Освобождает стековый фрейм

Возвращает управление вызывающей стороне

Код демонстрирует принцип создания массива B:

```
for i in range(len(A)):
    flag = True
    for j in range(i):
        if A[i] <= A[j]:
            flag = False
            break
    if flag:
        B.append(A[i])
```

io.asm

Модуль ввода-вывода, обеспечивающий работу с файлами, парсинг данных и форматированный вывод.

Основные методы:

remove_newline - удаляет символ новой строки из строки:

- Ищет символ '\n' в строке

- Заменяет его на нулевой терминатор

- Модифицирует строку на месте

read_file - читает содержимое файла в буфер:

- Открывает файл (системный вызов 1024)

- Читает данные в буфер (системный вызов 63)

- Закрывает файл (системный вызов 57)

- Возвращает количество прочитанных байт или -1 при ошибке

parse_file_data - парсит данные массива из файла:

- Читает размер массива (первое число)

- Проверяет корректность размера (1-10)

- Последовательно парсит все элементы массива

- Возвращает размер массива или -1 при ошибке

parse_int - парсит целое число из строки:

- Пропускает ведущие пробелы

- Обрабатывает знак '-' для отрицательных чисел

- Последовательно обрабатывает цифры

- Возвращает число и указатель на следующий символ

print_array - форматированный вывод массива:

- Выводит элементы через пробел

- Не выводит пробел после последнего элемента

- Добавляет перевод строки в конце

test.asm

Модуль автоматизированного тестирования, проверяющий корректность работы программы с различными наборами данных.

Основные методы:

run_tests - запускает полный набор автоматических тестов:

- Выводит заголовок тестовой сессии

- Последовательно выполняет 6 тест-кейсов

- Разделяет тесты визуальными разделителями

- Использует макросы для вывода информации

run_test - выполняет один тест-кейс с полной проверкой:

Выводит размер массива N

Проверяет корректность размера через *check_n*

Выводит исходный массив A

Формирует массив B через *form_array_B*

Выводит результирующий массив B

Сравнивает ожидаемый и фактический размер B

Выводит результат теста (PASSED/FAILED)

check_n - проверяет корректность размера массива:

Проверяет что $1 \leq N \leq 10$

Возвращает 1 при корректном размере, 0 при некорректном

Тест 1

Массив A = []

N = 0

Ожидаемый результат: ошибка размера

Тест 2

Массив A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]

N = 11

Ожидаемый результат: ошибка размера

Тест 3

Массив A = [2, 2, 2]

N = 3

Ожидаемый массив B = [2]

Ожидаемый размер B = 1

Тест 4

Массив A = [5, 4, 3, 2, 1]

N = 5

Ожидаемый массив B = [5]

Ожидаемый размер B = 1

Тест 5

Массив A = [1, 2, 3, 4, 5]

N = 5

Ожидаемый массив B = [1, 2, 3, 4, 5]

Ожидаемый размер B = 5

Тест 6

Массив A = [1, 2, 5, 4, 3]

N = 5

Ожидаемый массив B = [1, 2, 5]

Ожидаемый размер B = 3

Результаты работы:

Тестирование:

Программа автоматически запускает тесты для наглядной проверки корректности кода

```
=== AUTOMATED TESTS ===
-----
N = 0
Error: invalid array size
Expected size: 0
Test PASSED
-----
N = 11
Error: invalid array size
Expected size: 0
Test PASSED
-----
N = 3
A = 2 2 2
B = 2
Expected size: 1
Actual size: 1
Test PASSED
-----
N = 5
A = 5 4 3 2 1
B = 5
Expected size: 1
Actual size: 1
Test PASSED
-----
N = 5
A = 1 2 3 4 5
B = 1 2 3 4 5
Expected size: 5
Actual size: 5
Test PASSED
-----
N = 5
A = 1 2 5 4 3
B = 1 2 5
Expected size: 3
Actual size: 3
Test PASSED
-----

=== USER MODE ===
Enter file path: |
```

ВЫВОД: Все тесты пройдены!

Интерактивная часть:

Программа просит пользователя ввести путь к файлу (полный), при некорректном вводе повторить ввод до тех пор, пока файл не будет корректен для обработки (в папке проекта есть 4 файла для тестирования программы)

Тест 1. Несуществующий файл

```
=== USER MODE ===
Enter file path: noname.txt
ERROR: The file cannot be opened!

Enter file path:
```

Тест 2. test_1.txt

Данные файла:

0

Результат тестирования:

ERROR: Invalid array size (allowed size 1-10)!

Enter file path:

Тест 3. test_2.txt

Данные файла:

11

1 2 3 4 5 6 7 8 9 10 11

Результат тестирования:

ERROR: Invalid array size (allowed size 1-10)!

Enter file path:

Тест 4. test_3.txt

Данные файла:

5

1 0 2 6 5

Результат тестирования:

N = 5

A: 1 0 2 6 5

B: 1 2 6

-- program is finished running (0) --

Тест 5. test_4.txt

Данные файла:

7

3 -2 8 -5 12 -1 7

Результат тестирования:

N = 7

A: 3 -2 8 -5 12 -1 7

B: 3 8 12

-- program is finished running (0) --

ВЫВОД: Программа успешно прошла все тесты, связанные с обработкой данных файла.